



Sistemas Distribuidos I (75.74)

Modelado de Cómputo Distribuido

MapReduce, Tiempo y Relojes

Docentes

- Pablo D. Roca
- Gabriel Robles
- Franco Barreneche

- Tomás Nocetti
- Nicolás Zulaica



● MapReduce

- Ejemplos: WordCount / WordFrequencies / Union / Intersect / Join
- Tiempo y relojes - Relojes Físicos
- Tiempo y relojes - Relojes Lógicos



MapReduce | Motivaciones

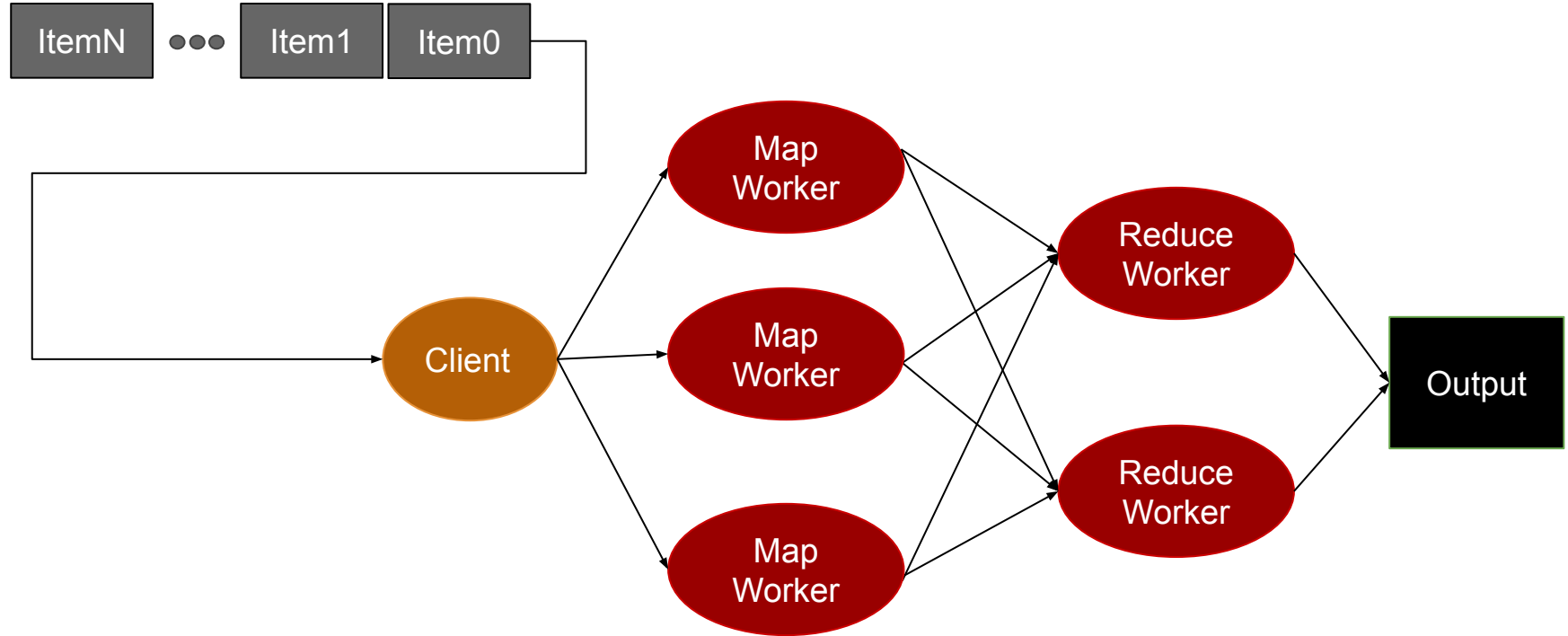
- Programación tradicional: ejecutada en ambientes serializados
- **Parallel Computing:** Partir procesamiento en partes que puedan ser ejecutadas concurrentemente en múltiples cores
- **Desafío**
 - No todos los problemas pueden ser paralelizados
 - Concepto de camino crítico
 - Concurrencia sobre recursos implica sincronización y retención de procesos
- **Idea:**
 - Identificar **Tareas** que puedan ser ejecutadas en paralelo
 - Identificar **Grupos de datos** que puedan ser procesados en paralelo



MapReduce | Introducción

- Paradigma de *parallel computing*
- Desarrollado en 2004 por Google
- Ligeramente basado en la idea de funciones **map** y **reduce** de LISP
 - `map f[a,b,c] => [f(a), f(b), f(c)]`
 - `map sqrt[a,b,c] => [sqrt(a), sqrt(b), sqrt(c)]`
 - `reduce f[a,b,c] => f(a,b,c)`
 - `reduce sum[1,2,3] => sum(1, sum(2, sum(3, sum(NULL))))`
- Implementaciones
 - [Apache Hadoop](#)
 - [Amazon EMR](#)
 - [Google MapReduce \(for AppEngine\)](#)

MapReduce | Arquitectura Naive





- No existe dependencia entre los datos
- Datos pueden ser partidos en *chunks* del mismo tamaño
- Cada proceso pueden trabajar con un *chunk/shard*
- **Master**
 - Encargado de partir la data en #*chunks*
 - Envía ubicación de los *chunks* a los Workers
 - Recibe ubicación de los resultados de todos los Worker
- **Workers**
 - Recibe ubicación de los *chunks* del Master
 - Procesa el *chunk*
 - Envía el ubicación del resultado de procesamiento al Master



MapReduce | Función Map

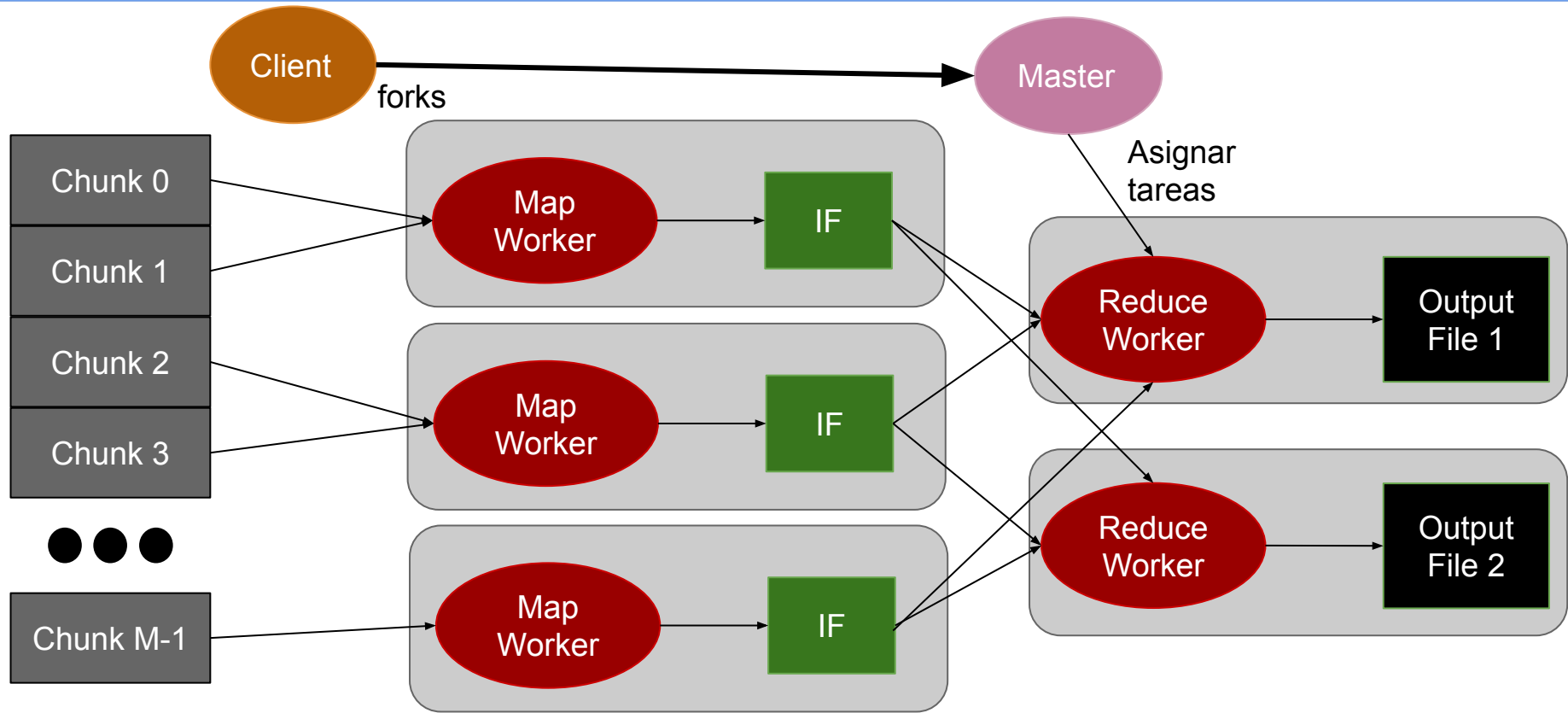
- **Map: (input shard) → intermediate(key/value pairs)**
 - Data es particionada automáticamente en **K chunks** y procesada por M workers ejecutando la función **map**
 - Función **Map** proporcionada por el usuario es ejecutada en todos los **chunks** de data
 - Usuario decide **cómo filtrar la data provista en los chunks**
 - Librería MapReduce agrupa todos los valores asociados con una misma key y envía ubicación de los datos al **Master Process**



MapReduce | Función Reduce

- **Reduce: intermediate(key/value pairs) → result files**
 - Función **Reduce** realiza una agregación de los datos para obtener un resultado final (result file)
 - Función **Reduce** es llamada por cada **Unique Key**
 - Realiza un *merge* de los datos recibidos para formar un set de datos menor
 - Función **Reduce** es distribuida particionando las keys en R Reduce workers
 - La cantidad de R workers es especificada por el usuario

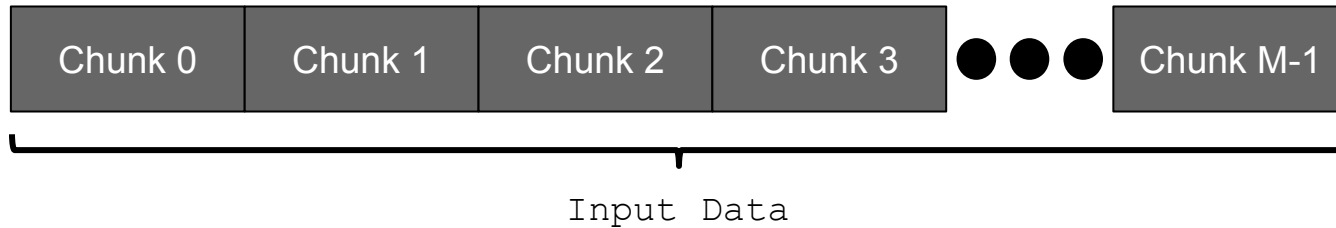
MapReduce | Arquitectura





MapReduce | Paso 1

- **Partir la datos datos de entrada en N *chunks***
 - Por lo general *chunks* de 64MB (configurable)





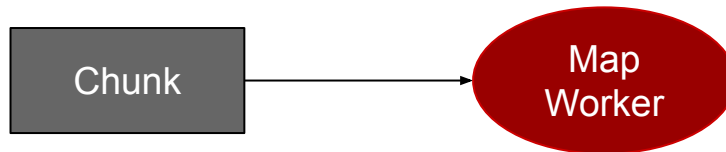
MapReduce | Paso 2

- **Fork de Procesos en Cluster**
 - 1 master: Actúa de Scheduler y Coordinador
 - Muchos Workers (Mappers y Reducers)
- **Mappers y Reducers**
 - Tantos Mappers como *Chunks*
 - R reducers (configurados por el usuario)



MapReduce | Paso 3

- **Map de Shards en Mappers**
 - Worker lee Input data
 - Filtra los datos recibidos en formato **key/value**
 - Ejecuta función provista por el usuario sobre cada par **key-value** que haya pasado el filtro
 - Por cada par produce un *valor intermedio*

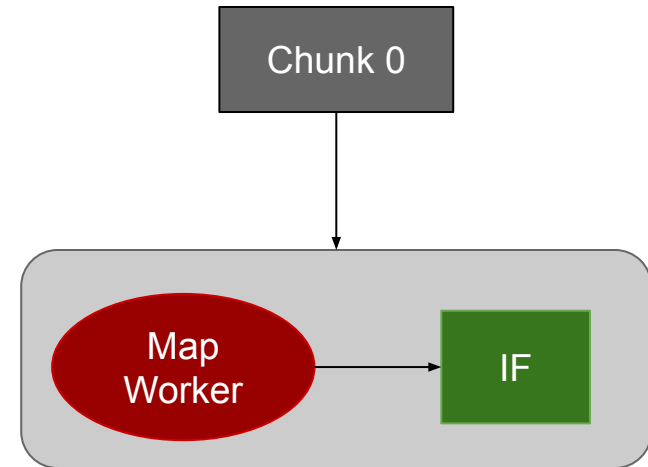




MapReduce | Paso 4a

- **Creación de Archivos Intermedios (Intermediate Files o IF)**

- Cada key/value intermedio producido es buffereado y escrito periódicamente en disco local
- La data es particionada en R regiones utilizando una función de particionamiento
- Notifica al **proceso master** cuando haya terminado de procesar el *chunk* de datos
- Envía al proceso **master** ubicación del archivo intermedio
- **Proceso master** envía ubicación de **IF** a **Reduce workers**





MapReduce | Paso 4b (Particionamiento)

- **Reducción de set de datos**

- La función Reduce del usuario será llamada una vez por unique key generada por los Map workers
- Keys/values deben ser agrupados por **Key** y se debe decidir que Reduce Worker se encargará de procesar cada key

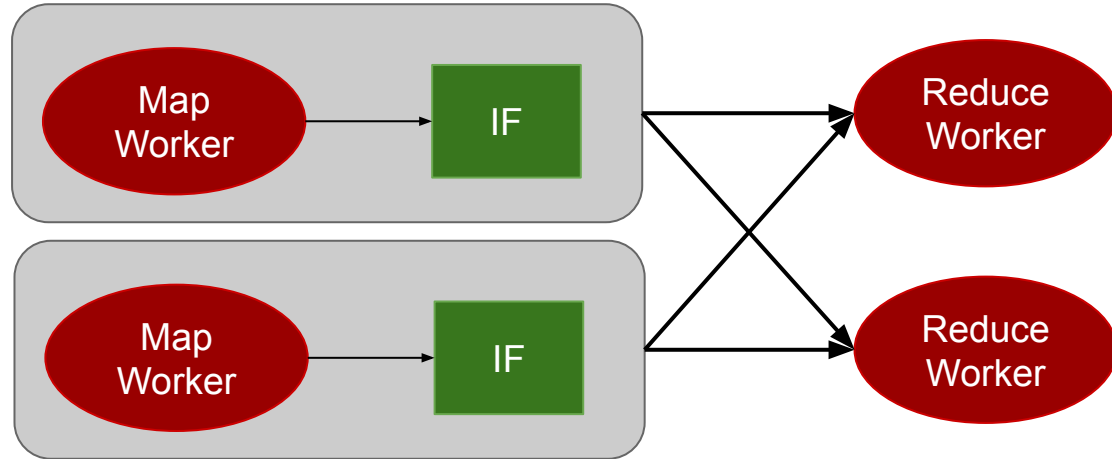
- **Función de Partición**

- Decide **cuál** de los R workers va a trabajar con cada **Key**
- Función default: **hash(key) mod R**
- **Map workers** particionan la data por **Keys** con esta función
- Cada Reduce Worker va a leer la partición que desea de cada **Map Worker**



MapReduce | Paso 5

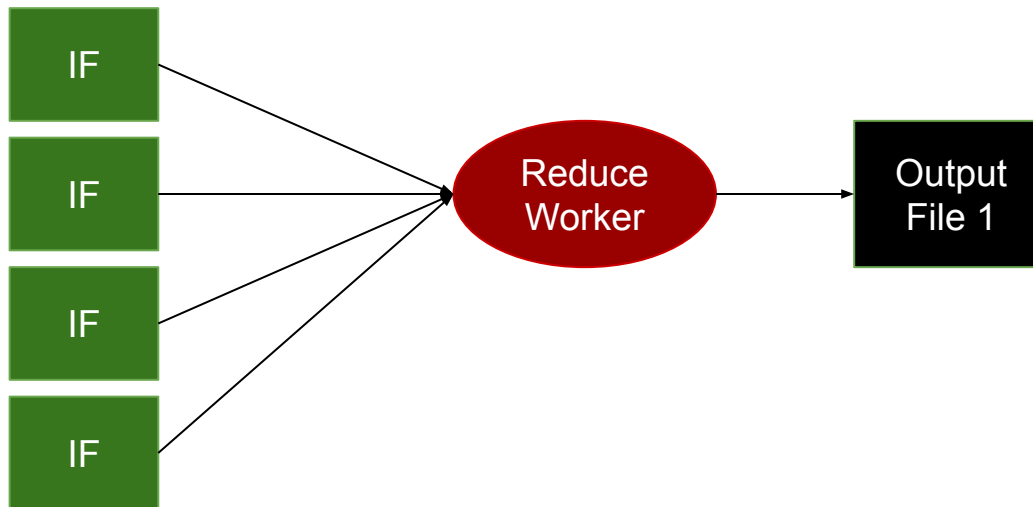
- **Reduce Workers** reciben ubicación de los archivos intermedios para la partición que deben procesar
 - Uso de RPC para leer data del disco local de los Mappers
- Cuando los Reduce Workers reciben la data intermedia de cada partición
 - Ordenan la data por **key**
 - Todas las ocurrencias de la misma **Key** son agrupadas





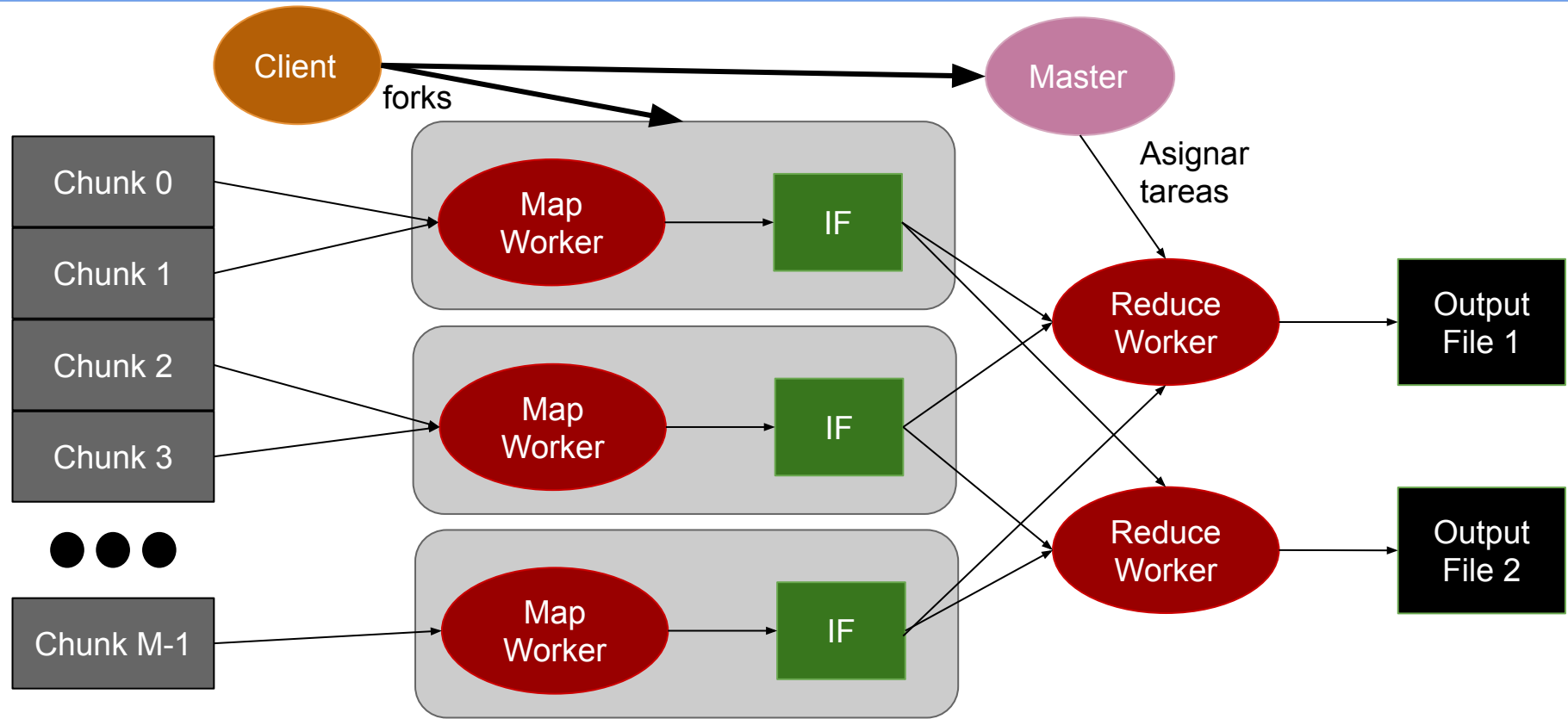
MapReduce | Paso 6

- Reduce worker lee data agrupada por Key...
 - Aplica función del usuario **Reduce** sobre cada set de datos
 - El output de la función es almacenado en un *output file*
 - Se despierta a User process y se le indica ubicación de los *output files*





MapReduce | Arquitectura



Agenda



- MapReduce
- **Ejemplos: WordCount / WordFrequency / Union / Intersect / Join**
- Tiempo y relojes - Relojes Físicos
- Tiempo y relojes - Relojes Lógicos



MapReduce | Word Count

- Contar la cantidad de ocurrencias de cada palabra dentro de un texto
- Ejercicio en [HackerRank](#)

```
map(string key, string value):
```

```
reduce(string key, list value):
```



MapReduce | Word Count

- Contar la cantidad de ocurrencias de palabras en un texto
- Ejercicio en [HackerRank](#)

```
map(string key, string value):  
    // key: document name  
    // value: document as a multiline string  
    for word w in value:  
        emitIntermediate(w, 1)  
  
reduce(string key, list value):  
    // key: word  
    // value: list of "1s" associated with the words  
    emit(key, len(value))
```



MapReduce | Word Frequency

- Contar la frecuencia de una palabra en un documento

```
map(string key, string value):
```

```
reduce(string key, list value):
```



MapReduce | Word Frequency

- Contar la frecuencia de una palabra en un documento

```
map(string key, string value):  
    // key: document name  
    // key: document as a multiline string  
    for word w in value:  
        emitIntermediate(w, 1)  
  
reduce(string key, list value):  
    // key: word  
    // value: list of "1s" associated with the words  
    emit(key, count(value) / totalWords)
```



MapReduce | Word Frequency

- Contar la frecuencia de una palabra en un documento

```
map(string key, string value):  
    // key: document name  
    // key: document as a multiline string  
    for word w in value:  
        emitAll("", 1)  
        emitIntermediate(w, 1)  
  
reduce(string key, list value):  
    // key: word  
    // value: list of "1s" associated with the words  
    totalWords = reduceAll("", sum)  
    emit(key, count(value) / totalWords)
```



- Dado N documentos, obtener palabras en uno o en ambos documentos

```
map(string key, string value):
```

```
reduce(string key, list value):
```




MapReduce | Union

- Dado N documentos, obtener palabras en algún documento

```
map(string key, string value):  
    // key: document name  
    // value: document as a multiline string  
    dict = {}  
    for word w in value:  
        if not word in dict:  
            dict[word] = 1  
            emitIntermediate(word, key)  
  
reduce(string key, list value):  
    // key: word  
    // value: dummy, list of documents where word appears  
    emit(key, key)
```



- Dado N documentos, obtener palabras que se encuentren en todos ellos

```
map(string key, string value):
```

```
reduce(string key, list value):
```



MapReduce | Intersect

- Dado N documentos, obtener palabras existentes en todos los docs

```
map(string key, string value):  
    // key: document name  
    // value: document as a multiline string  
    for word in value:  
        emitIntermediate(word, key)  
  
reduce(string key, list value): // key: word, value: list of repetitions  
    dictionary = {}  
    for v in value:  
        if not v in dictionary:  
            dictionary[v] = true  
    if len(dictionary.keys()) == amountDocuments:  
        emit(key, key)
```



MapReduce | Intersect

```
map(string key, string value):
    // key: document name - value: document as a multiline string
    dict = {}
    emitAll("", key)
    for word w in value:
        if not word in dict:
            dict[word] = key
            emitIntermediate(word, key)

reduce(string key, list value):
    amountDocuments = reduceAll("", (sort, uniq, len))
    dictionary = {}
    for v in value:
        if not v in dictionary:
            dictionary[v] = true
    if len(dictionary.keys()) == amountDocuments:
        emit(key, key)
```



MapReduce | Join

- Dados $S1 := [\text{shared_field}, \text{field_1}]$ y $S2 := [\text{shared_field}, \text{field_2}]$, obtener el conjunto final con la forma $S3 := [\text{shared_field}, \text{field_1}, \text{field_2}]$

```
map(string key, string value):
```

```
reduce(string key, list value): // key: join fields, value: tuples
```



MapReduce | Join

- Dados $S1 := [\text{shared_field}, \text{field_1}]$ y $S2 := [\text{shared_field}, \text{field_2}]$, obtener el conjunto final con la forma $S3 := [\text{shared_field}, \text{field_1}, \text{field_2}]$

```
map(string key, string value):  
    // key: set names as S1|S2|S3, value: multiline csv as "shared_field,field_X"  
    for line in value:  
        fields = line.split(',')  
        emitIntermediate(fields[0], (key, fields[1]))  
  
reduce(string key, list value): // key: join fields, value: tuples  
    data_fields = [key, None, None]  
    for set_data in value:  
        if set_data[0] == 'S1':  
            data_fields[1] = set_data[1]  
        else: // set_data[0] == 'S2':  
            data_fields[2] = set_data[1]  
    emit(key, ','.join(data_fields))
```



- MapReduce
- Ejemplos: WordCount / WordFrequencies / Union / Intersect / Join
- **Tiempo y relojes - Relojes Físicos**
- Tiempo y relojes - Relojes Lógicos



Magnitud para medir duración y separación de eventos.



Se lo puede definir mediante una variable monotónica creciente:

- En sistemas de computación será discreta
- No necesariamente vinculada con la hora de la vida real

El tiempo siempre avanza, nunca retrocede.



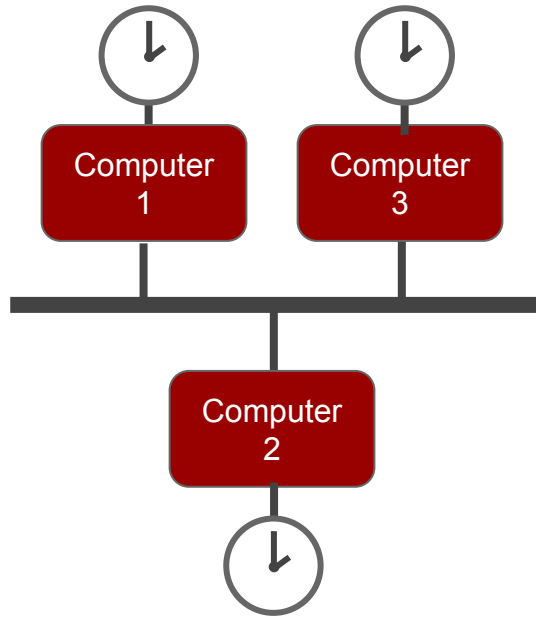
La medición del tiempo permite:

- Ordenar y sincronizar
- Marcar la ocurrencia de un suceso
 - **Timestamps:** puntos absoluto en la línea de tiempo
- Contabilizar duración entre sucesos
 - **Timespans:** intervalo en la línea de tiempo

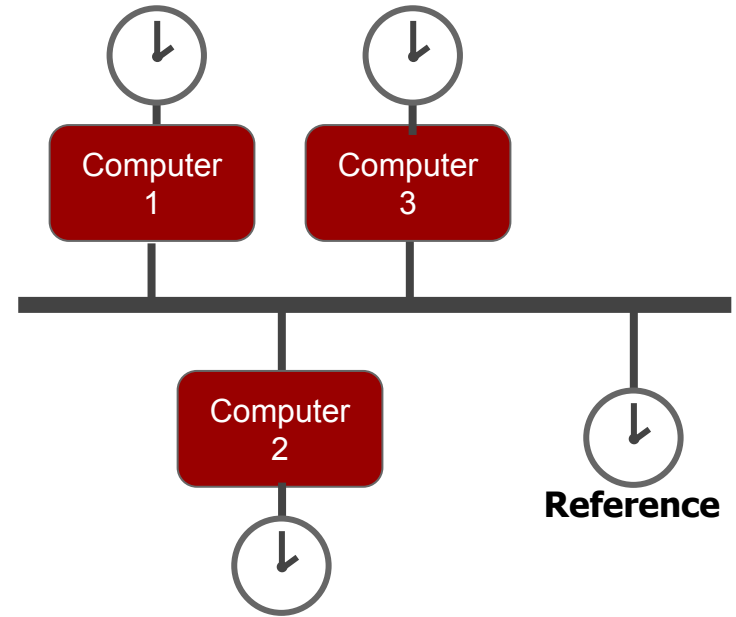
Relojes Físicos | Locales vs Globales



Locales



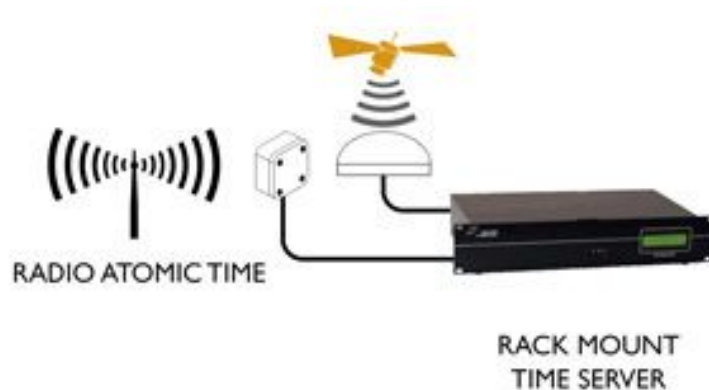
Locales + Global



Relojes Físicos



- Brindan la fecha y hora del día
- Pueden ser de cuarzo, atómicos, por GPS
- Los cambios de temperatura, presión y humedad descalibran relojes: *drift*





Relojes Físicos | Referencias Globales

GMT (Greenwich Mean Time)

- Basado en el tiempo de rot. Terrestre (t. astronómico)

UTC (Universal Time Coordinated)

- Basado en medición de relojes atómicos
- Dif. de +/-1 seg con GMT y recibe ajustes periódicos

GPS time (Global Positioning System time)

- Basado en relojes atómicos en la Tierra
- No recibe ajustes pero permite ajustar satélites

TAI (Temps Atomique International)

- 200 relojes atómicos en 70 países
- No recibe ajustes astronómicos

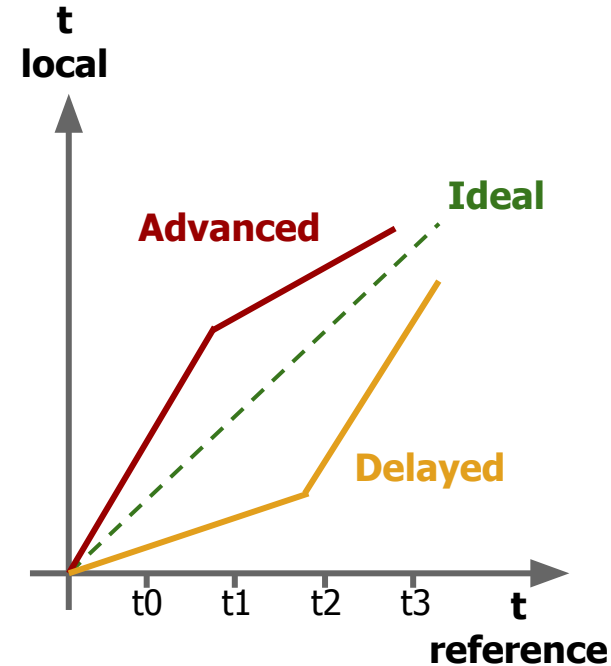


Source: wikipedia.org



Relojes Físicos | *Drift*

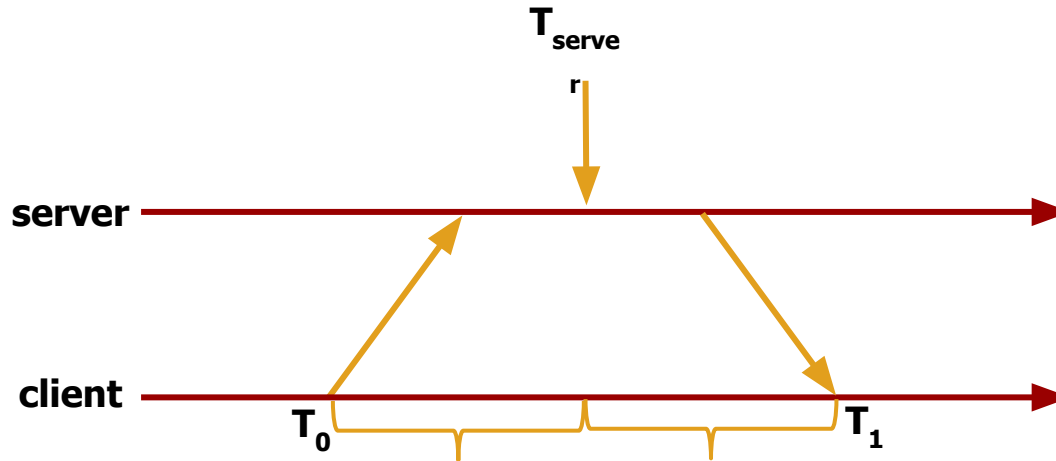
- Los relojes físicos no son confiables para su comparación por efecto del *drift*
- Hay que sincronizarlos periódicamente:
 - Medir el desvío respecto de un reloj de referencia (UTC, GPS, etc.)
 - Aplicar corrección o compensación lineal cambiando la frecuencia del reloj local
 - Nunca atrasar un reloj
- Hay que sincronizarlos al despertar la computadora





Relojes Físicos | *Drift - Algoritmo de Cristian*

- Realiza una compensación del delay existente al obtener la medida de tiempo
 - T_0 : Cliente envía request
 - T_1 : Cliente recibe respuesta
 - Hipótesis: *Delays en la red son constantes*



$$T_{new} = T_{server} + (T_1 - T_0) / 2$$

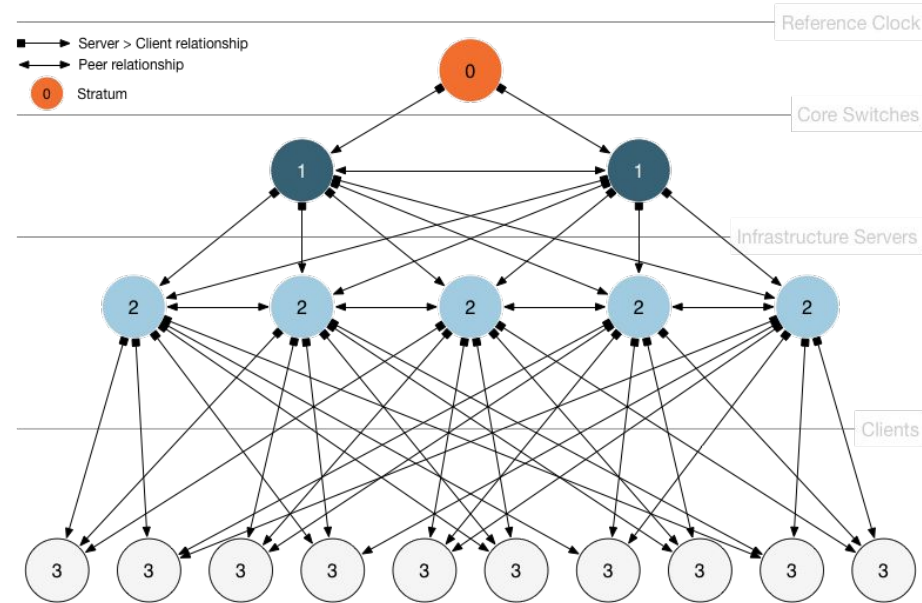


- **Sincronización**
 - Clientes (UTC) sincronizados aunque existan delays en la red
 - Análisis estadístico para filtrar data y obtener resultados de calidad
- **Alta disponibilidad**
 - Sobrevivir a caída a largas caídas de conectividad
 - Rutas redundantes
 - Servidores redundantes
- **Escalabilidad**
 - Gran número de clientes sincronizados de forma frecuente
 - Debe tener en cuenta efectos de drift



Basado en stratus (estratos)

- Estrato 0: Master clocks
- Estrato 1: Servidores conectados directamente a master clocks
- Estrato 2: Servidores sincronizados con servidores en estrato 1
- Estrato N: Servidores sincronizados con servidores en estrato N-1
- Servidores sincronizados entre sí con conexiones peer-to-peer
- Mensajes enviados de forma no *reliable* (UDP puerto 123)





- **Modo multicast/broadcast**
 - Usado en LANs de alta velocidad
 - Eficiente pero de baja precisión
- **Modo Cliente-Servidor (RPC)**
 - Grupos de aplicaciones se conectan formando un grupo
 - Aplicaciones entre sí no pueden sincronizarse
- **Modo Simétrico (Peer Mode)**
 - Peers sincronizados entre sí para proveer backup mutuo
 - Utilizado en estratos 1 y 2



- MapReduce
- Ejemplos: WordCount / WordFrequencies / Union / Intersect / Join
- Tiempo y relojes - Relojes Físicos
- **Tiempo y relojes - Relojes Lógicos**



Relojes Lógicos | Evento, estado, ocurre antes

Evento

- Suceso relativo al proceso P_i que modifica su estado

Estado

- Valores de todas las variables del proceso P_i en un momento dado

Relación 'ocurre antes' (*happened before*):

- Relación de causalidad entre eventos o estados tales que:
 - $a \rightarrow b$, si a, b pertenecen al mismo proceso P_i y a ocurre antes de b
 - $a \rightarrow b$, si a es un evento de P_i , b es un evento de P_j , a es el envío del mensaje ' m ' a P_j y b es la recepción del mensaje ' m ' desde P_i
 - $a \rightarrow c$, si $a \rightarrow b$ y $b \rightarrow c$ (transitividad)

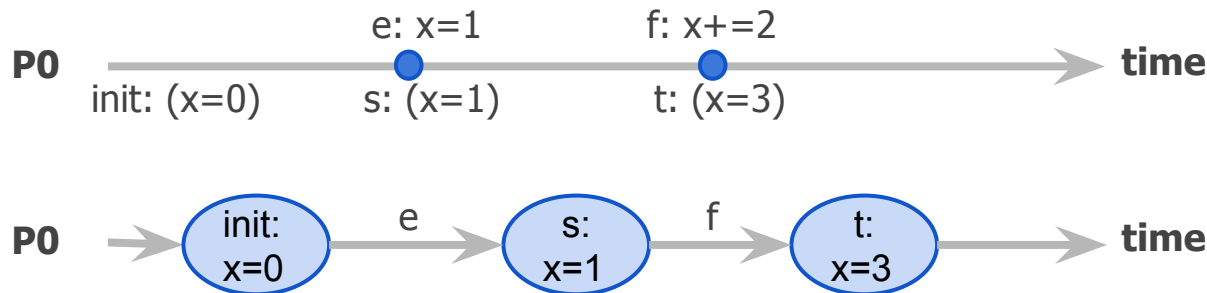


Relojes Lógicos | Definición

Dado S , el conjunto de todos los estados locales posibles del sistema y $\rightarrow$ la relación temporal de implicancia 'ocurre antes' (o *happened before*), un reloj lógico es una función C monotónica creciente que mapea estados con un número Natural y garantiza:

$$\forall s, t \in S : s \rightarrow t \Rightarrow C(s) < C(t)$$

Ej.:



$$s \rightarrow t$$
$$C(s) < C(t)$$



Relojes Lógicos | Algoritmo de Lamport

Pi:

var:

$c := 0$

send event (s, send, t):

//include t.c in msg

//send msg

$t.c := s.c + 1$

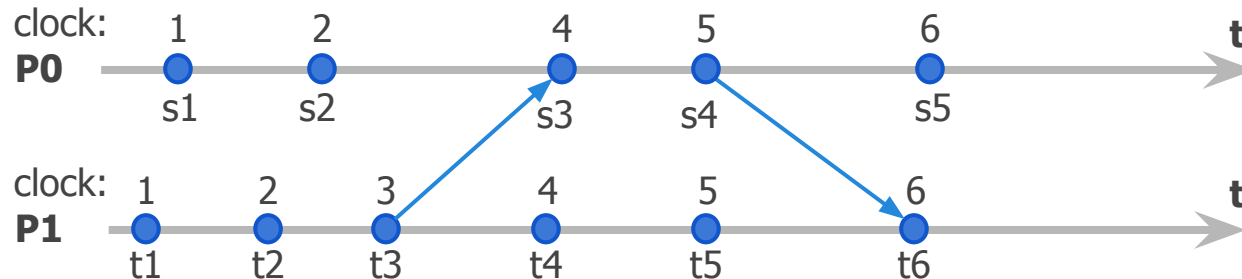
receive event (s, receive(u), t):

//receive msg from u

$t.c := \max(s.c, u.c) + 1$

internal event (s, internal, t):

$t.c := s.c + 1$





Relojes Lógicos | Inconvenientes

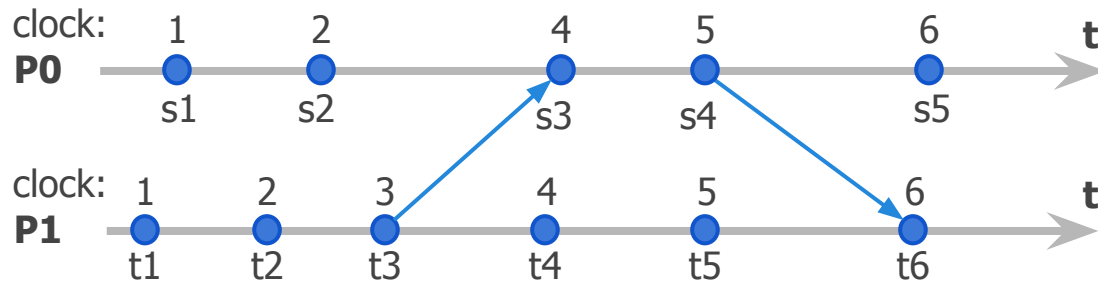
Los relojes de Lamport garantizan:

$$s \rightarrow t \Rightarrow C(s) < C(t)$$

pero:

$$C(s) < C(t) \not\Rightarrow s \rightarrow t$$

Ej.:



$$C(t1) < C(s4) \\ t1 \rightarrow s4$$

$$C(s1) < C(t4) \\ s1 \not\rightarrow t4$$

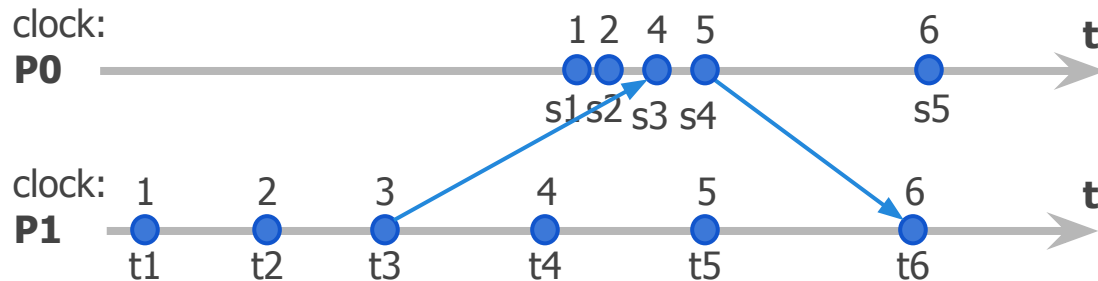


Relojes Lógicos | Inconvenientes (II)

El reloj de Lamport captura muy poca información sobre dos eventos:

1. progreso dentro de un proceso
2. progreso durante envíos de mensajes entre procesos

pero se pierde la información de qué evento ha ocurrido se pierde al almacenar un único escalar.





Relojes Lógicos | Corolario

Conociendo los relojes de Lamport se pueden detectar eventos paralelos

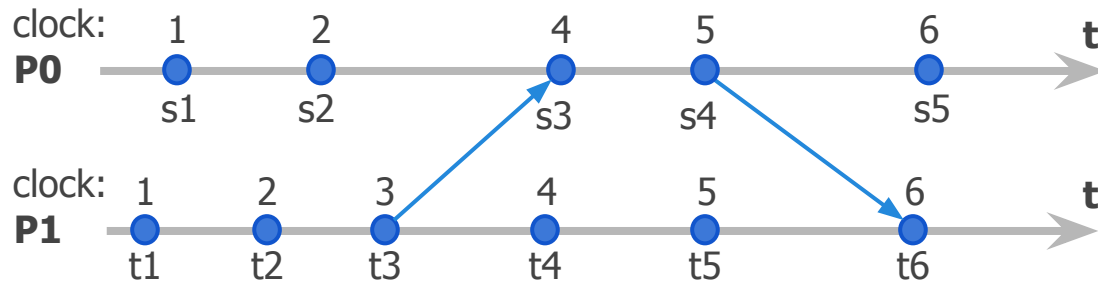
Ej:

- $c(s_2) = 2$
- $c(t_2) = 2$

$$C(s_2) \not\prec C(t_2) \Rightarrow s_2 \nrightarrow t_2$$

$$C(t_2) \not\prec C(s_2) \Rightarrow t_2 \nrightarrow s_2$$

$$s_2 \parallel t_2$$





Vector de Relojes | Definición

Un vector de relojes es el mapeo de todo estado del sistema compuesto por k procesos, con un vector de k números Naturales y garantiza:

$$\forall s, t \in S : s \rightarrow t \Leftrightarrow s.v < t.v$$

donde $s.v$ y $t.v$ son los vectores de k componentes para los estados s y t respectivamente y

$$s.v < t.v \Leftrightarrow \forall k : s.v[k] \leq t.v[k] \wedge \\ \exists j : s.v[j] < t.v[j]$$



Vector de Relojes | Implementación

Pi:

var:

$v[i] := 1$

$v[j] := 0$, con $i \neq j$

send event (s, send, t):

//include t.v in msg and send

$t.v := s.v$

$t.v[i] := t.v[i] + 1$

receive event (s, receive(u), t):

//receive msg from u

for $j := 1$ to N :

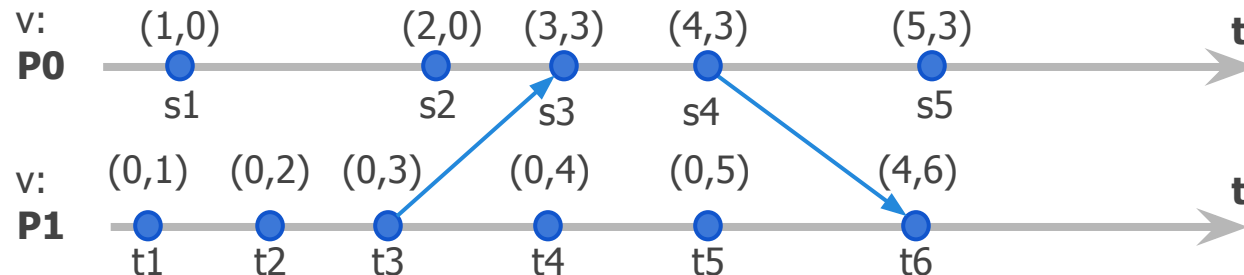
$t.v[j] := \max(s.v[j], u.v[j])$

$t.v[i] := t.v[i] + 1$

internal event (s, internal, t):

$t.v := s.v$

$t.v[i] := t.v[i] + 1$





Vector de Relojes | Implementación

Pi:

var:

$v[i] := 1$

$v[j] := 0$, con $i \neq j$

send event (s, send, t):

//include t.v in msg and send

$t.v := s.v$

$t.v[i] := t.v[i] + 1$

receive event (s, receive(u), t):

//receive msg from u

for $j := 1$ to N :

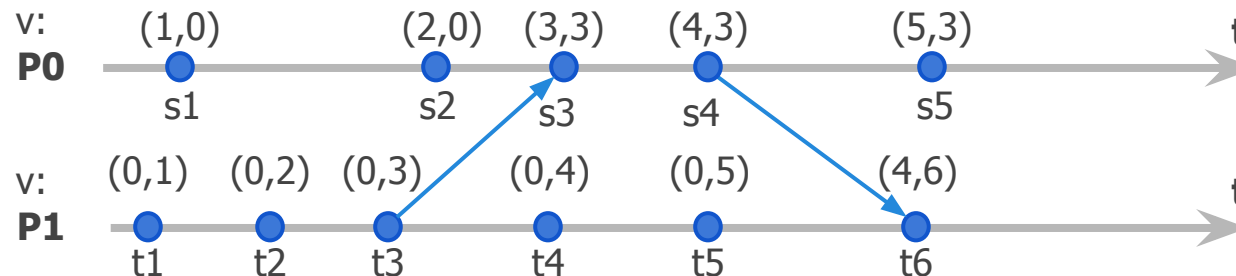
$t.v[j] := \max(s.v[j], u.v[j])$

$t.v[i] := t.v[i] + 1$

internal event (s, internal, t):

$t.v := s.v$

$t.v[i] := t.v[i] + 1$



$V(t1) < V(s4)$
 $(0,1) < (4,3)$
 $i=0 \Rightarrow 0 < 4$
 $i=1 \Rightarrow 1 < 3$
 $\Rightarrow t1 \rightarrow s4$



Vector de Relojes | Implementación

Pi:

var:

$v[i] := 1$

$v[j] := 0$, con $i \neq j$

send event (s, send, t):

//include t.v in msg and send

$t.v := s.v$

$t.v[i] := t.v[i] + 1$

receive event (s, receive(u), t):

//receive msg from u

for $j := 1$ to N :

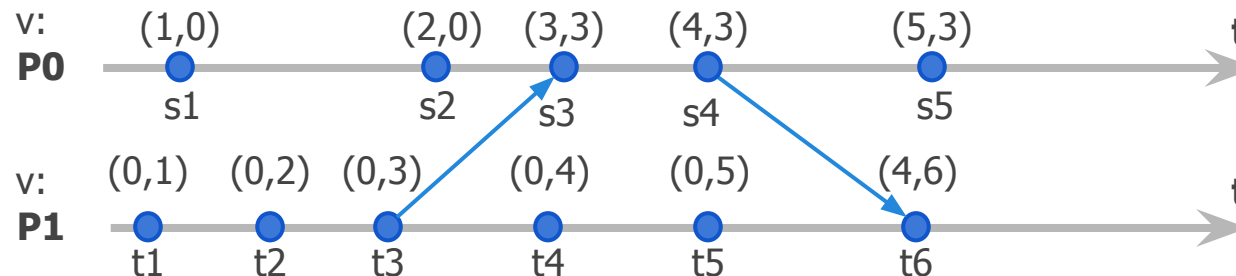
$t.v[j] := \max(s.v[j], u.v[j])$

$t.v[i] := t.v[i] + 1$

internal event (s, internal, t):

$t.v := s.v$

$t.v[i] := t.v[i] + 1$



$V(t4) < V(s1)$
 $(0,4) < (1,0)$
 $i=0 \Rightarrow 0 < 1$
 $i=1 \Rightarrow 4 < 0$
 $\Rightarrow t1 \neq s4$



Bibliografía

- MapReduce
 - <http://static.googleusercontent.com/media/research.google.com/en//archive/papers/mapreduce-sigmetrics09-tutorial.pdf>
- HackerRank - Distributed Systems
 - <https://www.hackerrank.com/domains/distributed-systems>, Ejercicios 'Relational MapReduce Patterns' y 'MapReduce Tutorials'
- McCool M., Robison A. D., Reinders J., Structured Parallel Programming Patterns for Efficient Computation, 2012, Elsevier-Morgan Kaufmann.
 - Capítulo 4.1 - Map
 - Capítulo 5.1 - Reduce
- Garg, V., Elements of Distributed Computing, 1st. Ed. Wiley IEEE Press, 2002.
 - Capítulo 2: Model of a computation
 - Capítulo 3: Logical Clocks
- P. Verissimo, L. Rodriguez: Distributed Systems for Systems Architects, Kluwer Academic Publishers, 2001.
 - Capítulo 2.1: Naming and addressing
 - Capítulo 2.4: Group Communication
 - Capítulo 2.5: Time and Clocks
- IPSES Scientific Electronics - Standard of Time Definition
 - <https://www.ipses.com/eng/In-depth-analysis/Standard-of-time-definition>